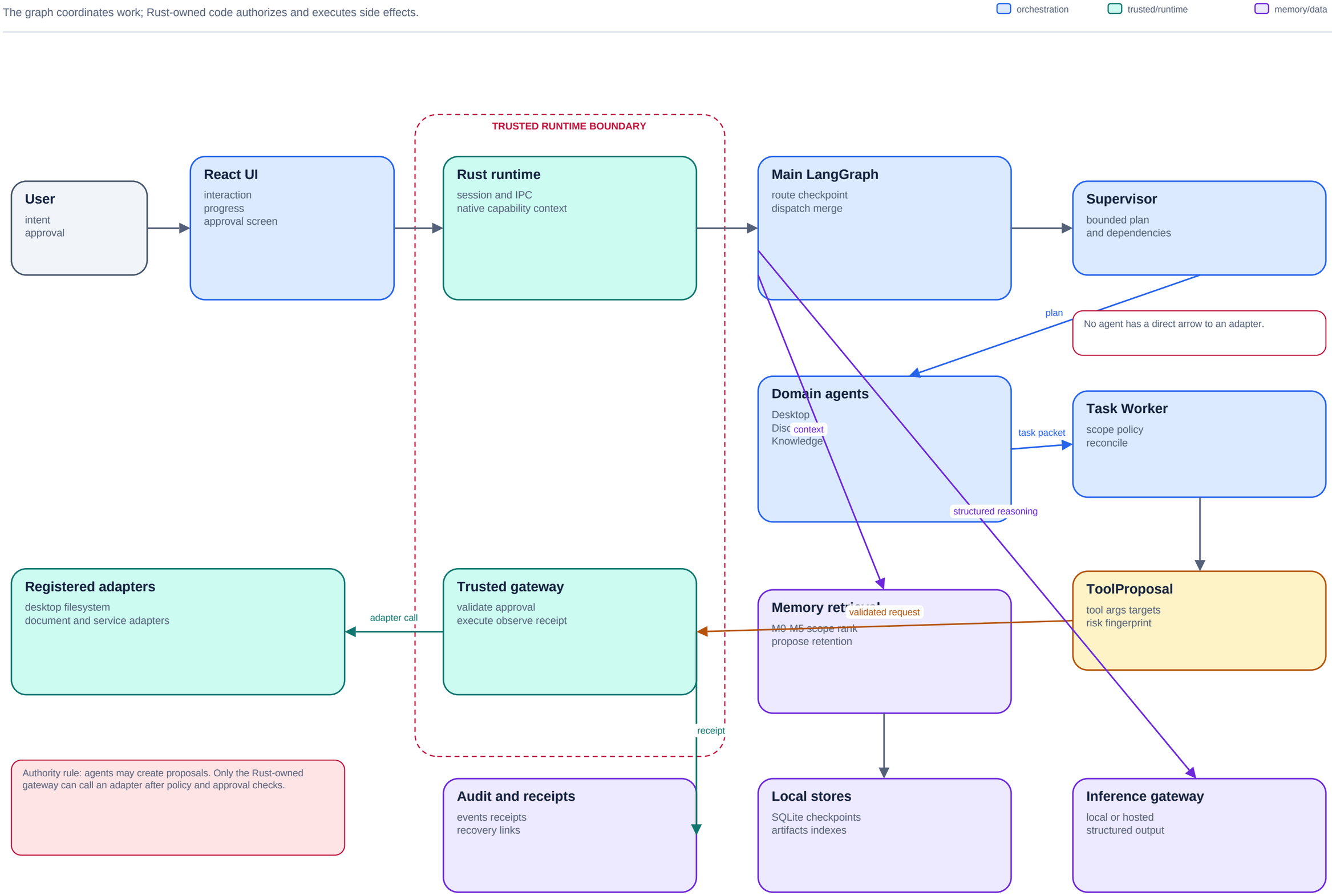
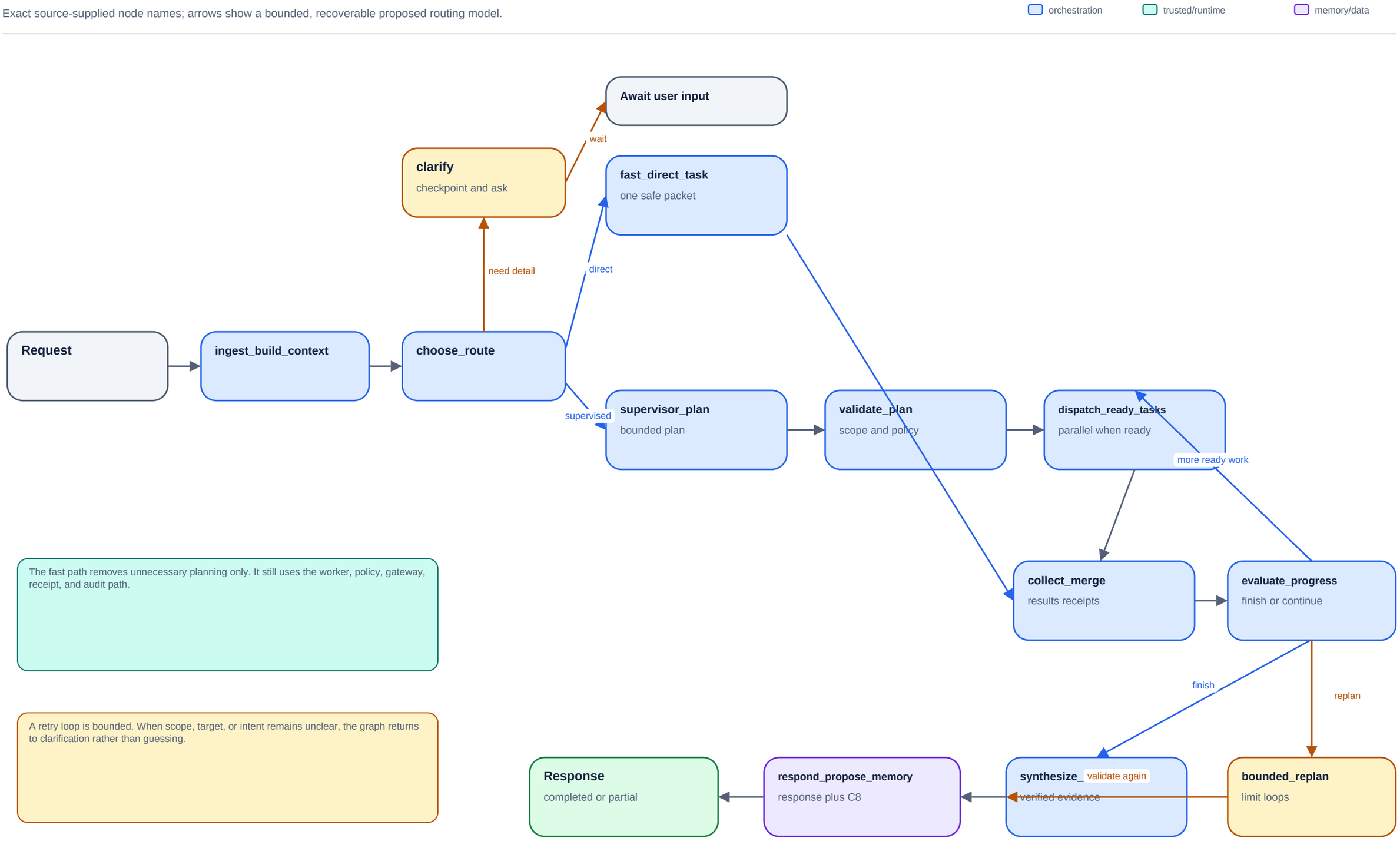
# D1  Overall system and trust boundaries

The graph coordinates work; Rust-owned code authorizes and executes side effects.



# D2  Main LangGraph execution flow

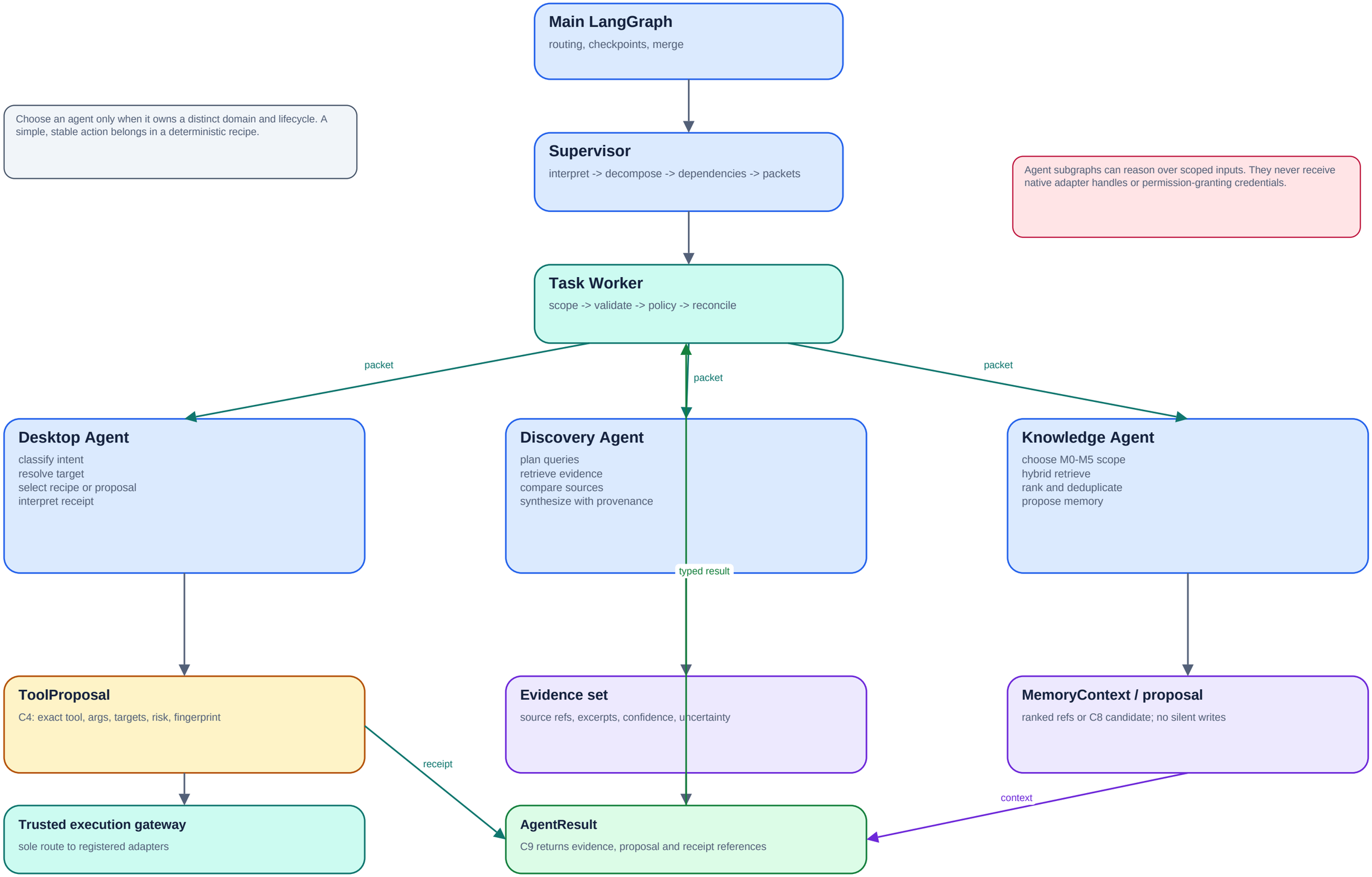
Exact source-supplied node names; arrows show a bounded, recoverable proposed routing model.



# D3  Agent hierarchy and bounded subgraphs

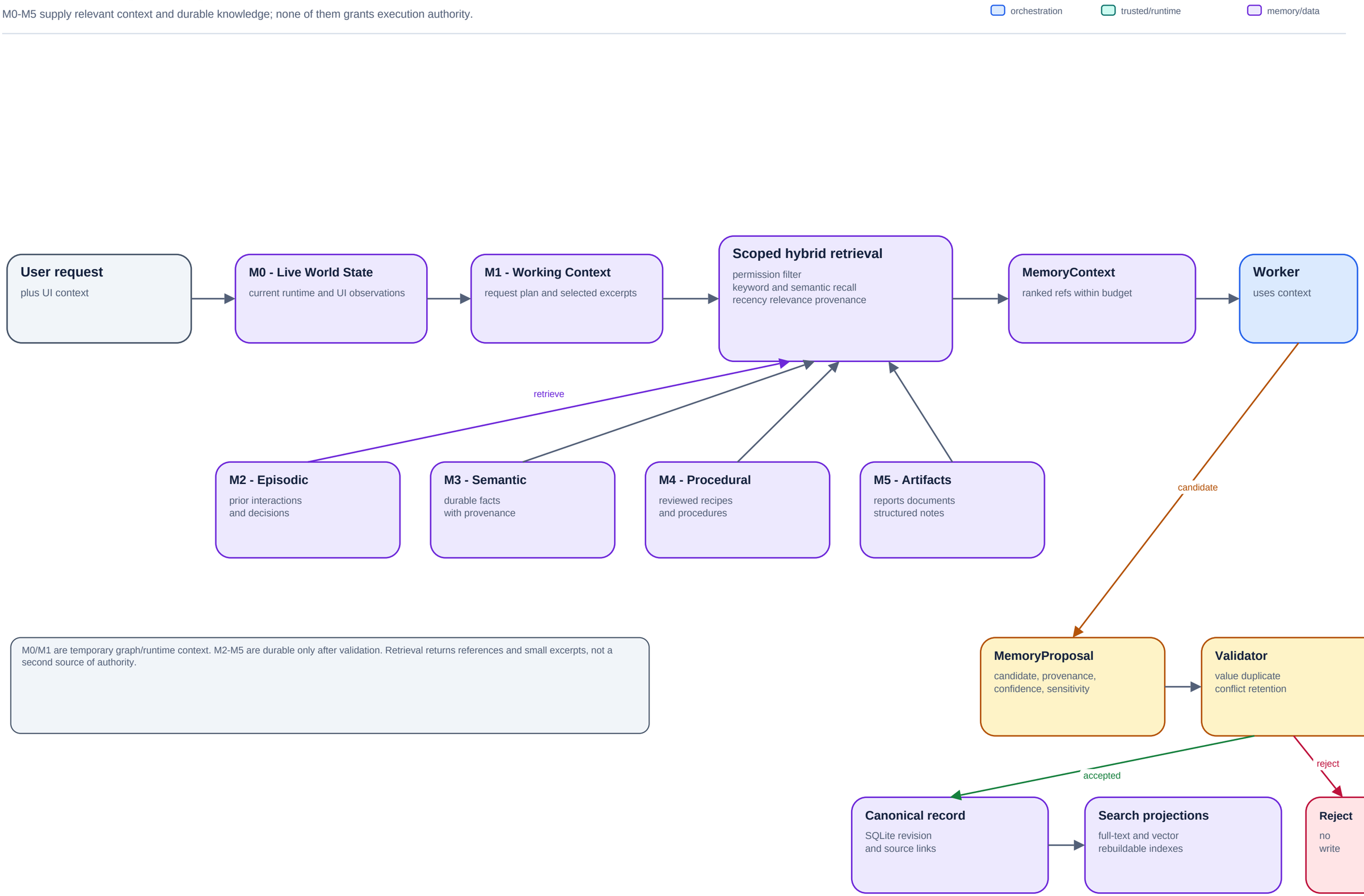
Specialization improves reasoning, but authority remains in the worker and trusted gateway.

orchestration
trusted/runtime
memory/data



# D4  Memory lifecycle and tier selection

M0-M5 supply relevant context and durable knowledge; none of them grants execution authority.

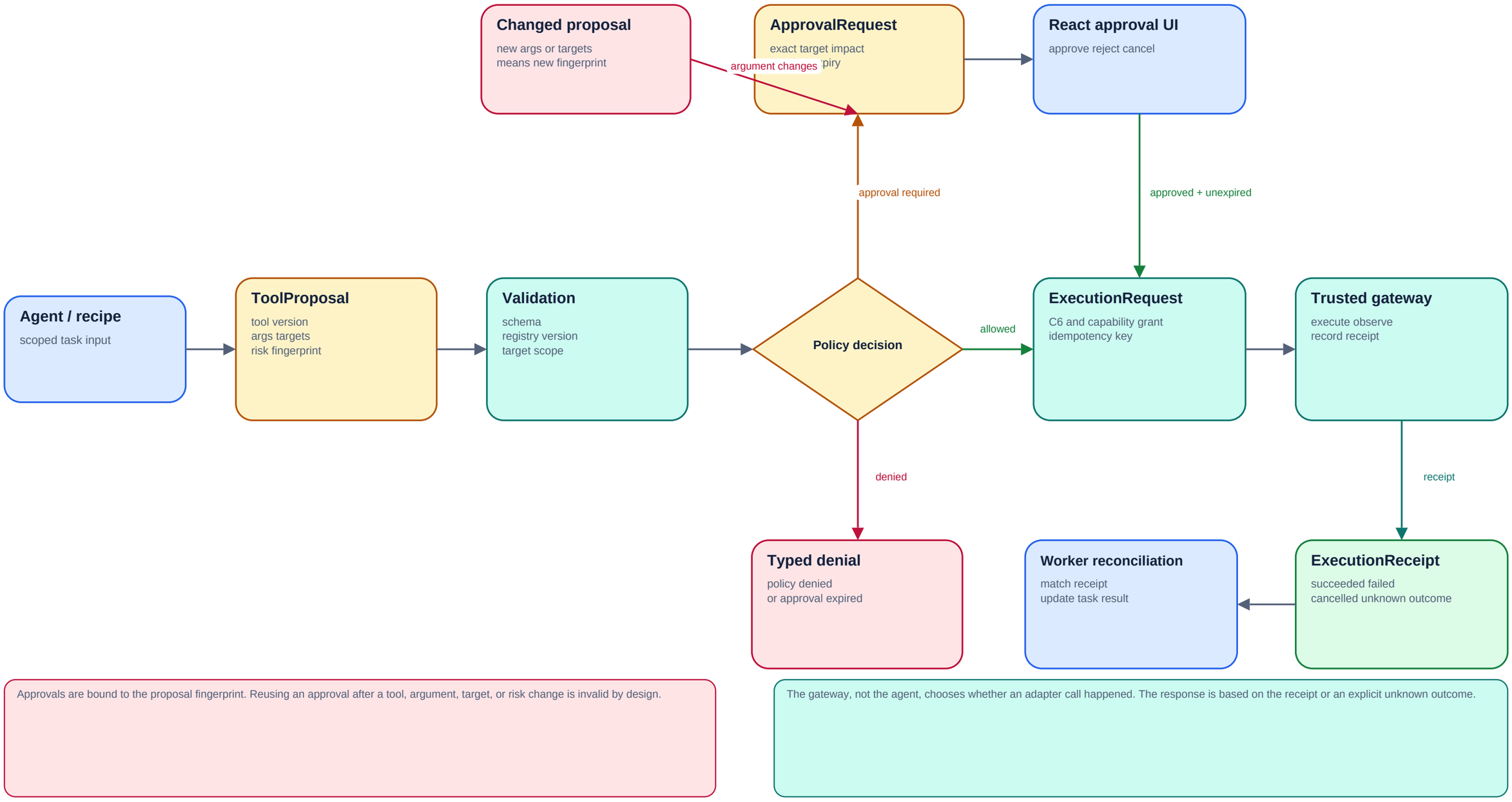



M0/M1 are temporary graph/runtime context. M2-M5 are durable only after validation. Retrieval returns references and small excerpts, not a second source of authority.

# D5  Tool approval and trusted execution

A proposal becomes an execution request only after exact validation, policy, and any required human approval.

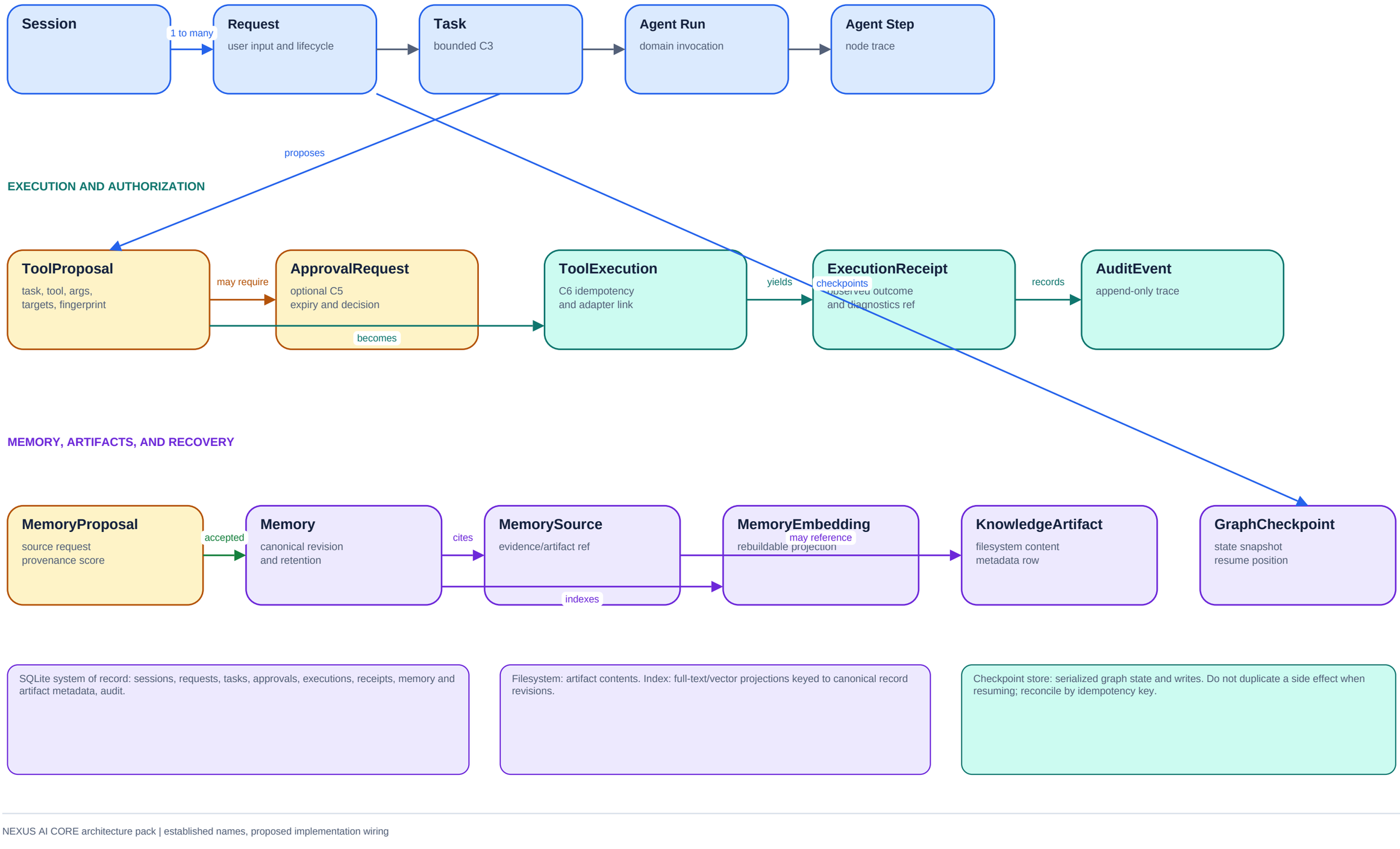
orchestration | trusted/runtime | memory/data




Approvals are bound to the proposal fingerprint. Reusing an approval after a tool, argument, target, or risk change is invalid by design.

The gateway, not the agent, chooses whether an adapter call happened. The response is based on the receipt or an explicit unknown outcome.

# D6  Persistence and relationship model

Proposed normalized local-first records. SQLite holds canonical metadata; artifacts and indexes live beside it.

orchestration  trusted/runtime  memory/data